Un experto que resuelve problemas debe estar dotado con dos cualidades incompatibles, una inquieta imaginación y una paciencia tenaz.

Howard W. Eves

Tema 5: Sistemas secuenciales.

5.1: Introducción.

5.2: Sistemas secuenciales asíncronos.

- 5.2.1: Biestables asíncronos.

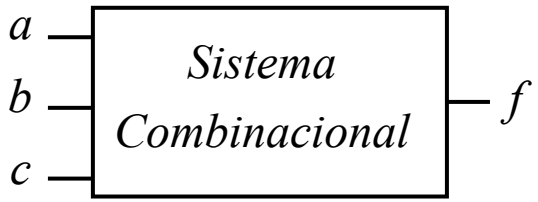
5.3: Sistemas secuenciales síncronos.

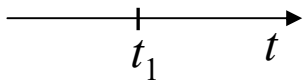
- 5.3.1: Biestables síncronos.
- 5.3.2: Análisis y síntesis de sistemas secuenciales síncronos.
  - Modelo de *Moore*.
- 5.3.3: Bloques funcionales síncronos
  - 5.3.3.1: Contadores.
  - 5.3.3.2: Registros.

- Los Sistemas Digitales se pueden clasificar en dos grupos, de acuerdo con su comportamiento:

$$\textit{Sistemas Digitales}\begin{cases} \cdot\ \textit{Sistemas combinacionales.} \\ \cdot\ \textit{Sistemas secuenciales}\begin{cases} \textit{asíncronos} \\ \textit{síncronos} \end{cases} \end{cases}$$

- Los *sistemas combinacionales* se caracterizan porque el valor lógico de sus salidas, en cualquier instante de tiempo *t* que se considere, *sólo* depende del valor lógico de sus entradas en dicho instante de tiempo *t*.





- Los *sistemas secuenciales* (*modelo de Moore*) se caracterizan porque el valor lógico de sus salidas, en cualquier instante de tiempo $t = t_1$ que se considere, dependen del *estado interno* del sistema en $t = t_1$.

El siguiente valor del estado interno de un sistema secuencial (*modelo de Moore*) depende del valor que tengan en el presente las *entradas* del sistema y el estado interno (ver más adelante la realimentación en el diagrama de bloques del *modelo de Moore*).

Ejemplos:

- Circuito de control de un ascensor.
- Circuito de control del cambio de canal de una T.V.

Nota: los sistemas secuenciales necesitan guardar información, mientras que los sistemas combinacionales no lo hacen.

▪ El *estado interno* (Q) de un sistema secuencial se representa por medio de un número binario que guardan unas variables lógicas denominadas *variables de estado*. Su valor, en cualquier instante de tiempo t que se considere, representa información acerca del pasado del sistema de modo que,

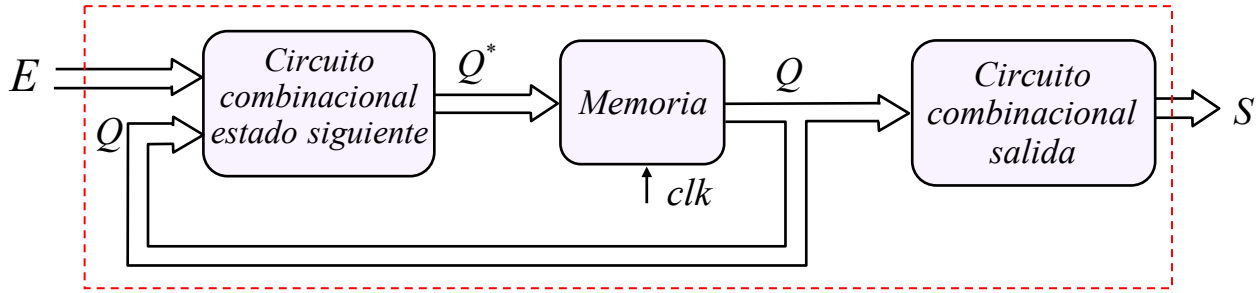
_ permite establecer el valor de las salidas del sistema en el presente (*modelo de Moore*)

$$S = f_{Moore}(Q)$$

_ junto con el valor de las entradas del sistema en el presente, permite establecer el valor del siguiente (próximo, futuro) estado interno del sistema

$$Q^* = g_{Moore}(E, Q)$$

$$Q^*_{t+\Delta t} = g_{Moore}(E_t, Q_t)$$



Las *variables de estado* se caracterizan por lo siguiente:

- ✓ Son variables lógicas que crea el propio sistema secuencial.

- ✓ Los valores del estado interno (Q) que representan no corresponden <u>necesariamente</u> con valores de magnitudes físicas conocidas.

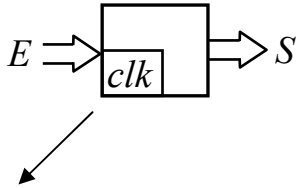
- ✓ La codificación en binario de los distintos estados internos del sistema no es única ≡ los distintos valores que guardan las variables de estado no son únicos.

- ✓ Los valores que representan n variables de estado siempre van a ser números binarios de n bits.

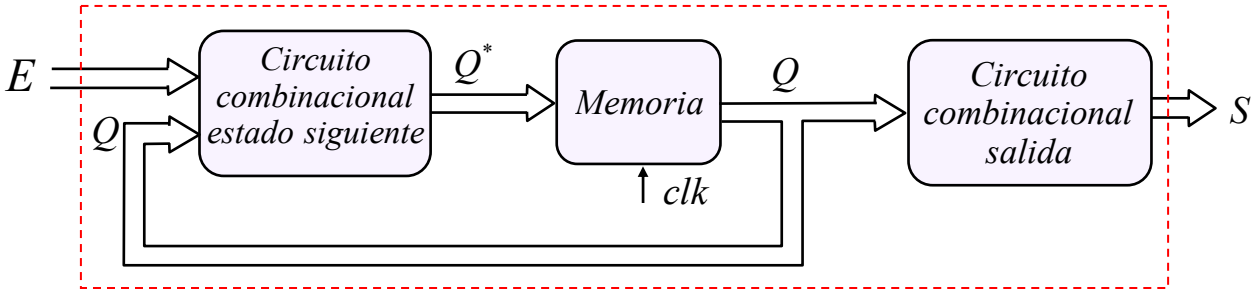
- ✓ Un sistema secuencial con n variables de estado tiene como mucho 2^n estados distintos.

número de estados finito ⇒ *sistemas* (*o máquinas*) *con un número finito de estados*

Los sistemas secuenciales se diseñan (y se analizan) siguiendo dos estructuras o modelos distintos, denominados modelos de *Mealy* y de *Moore* (en esta asignatura sólo vemos el modelo de *Moore*)



Modelo de Moore



E: entradas del sistema secuencial.

S: salidas del sistema secuencial.

clk: señal de reloj que sincroniza el funcionamiento del sistema secuencial.

Q: estado interno (*conjunto de variables de estado*) en el presente.

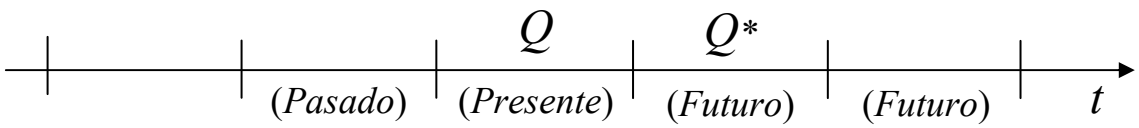
Q^*: próximo o siguiente estado interno del sistema.

Nomenclatura en esta asignatura relativa a sistemas secuenciales

Q: *representa el valor del estado interno del sistema en el presente* (≡ *número binario de* n *dígitos que guardan las* n *variables de estado en el presente*).

Q^*: *representa el valor que va a tener el estado interno en el futuro que está a continuación del presente* (*es el próximo número binario que guardarán las variables de estado*)

Nomenclatura: $Q_t \equiv Q \qquad Q_{t+T} \equiv Q_{t+1} \equiv Q^*$

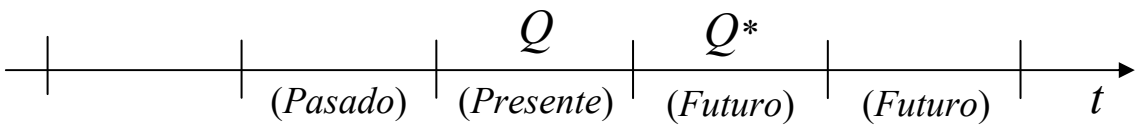


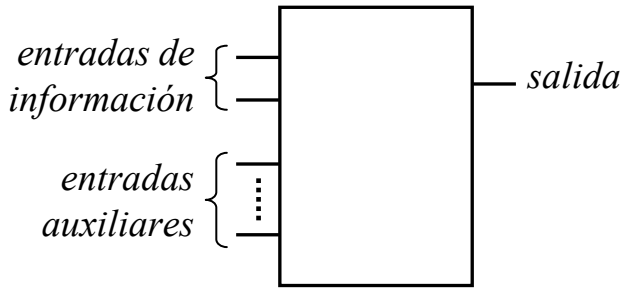
Nomenclatura en esta asignatura relativa a biestables

Q : *representa el valor que guarda un biestable en el presente.*

Q^* : *representa el valor que va a guardar un biestable en el futuro que está a continuación del presente.*

Nomenclatura: $Q_t \equiv Q \qquad Q_{t+T} \equiv Q_{t+1} \equiv Q^*$





Dispositivos de memoria: Biestables.

Características:

- Un *biestable* es un circuito digital muy sencillo que guarda (memoriza) 1 bit de información.

- Básicamente, un *biestable* posee: 1 salida (Q), 1 ó 2 entradas de información (según el tipo de *biestable* que se considere) y, en algunos casos, 1 ó 2 entradas auxiliares.

- Las entradas de información sirven para indicarle al *biestable* el valor lógico que debe memorizar (0 ó 1).

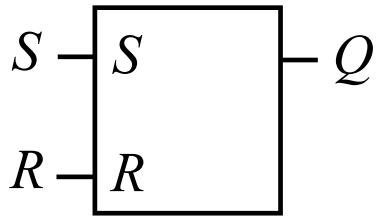
- La salida proporciona en todo momento el valor que está guardando el *biestable*.

- Las entradas auxiliares (si las hay) permiten controlar su funcionamiento

- Una posible clasificación de los biestables es la siguiente:

Biestables
- *Asíncronos* (*Latches*)
  - *Sin entrada de habilitación*: ***SR*** (*D*, *JK*, *T*)
  - *Con entrada de habilitación*: ***D*** (*SR*, *JK*, *T*)
- *Síncronos* (*Flip-flops*)
  - *Disparados por flanco* (*edge triggered*): ***D***, *JK*, *T*, (*SR*, *PQ*)

Latch S-R:



✓ Es el más sencillo de <u>todos</u> los biestables

✓ Tiene dos entradas (de información):

S ≡ *set* → *sirve para indicarle al biestable* (*latch*) *que guarde un* 1

R ≡ *reset* → *sirve para indicarle al biestable* (*latch*) *que guarde un* 0

✓ Tiene una salida (*Q*)

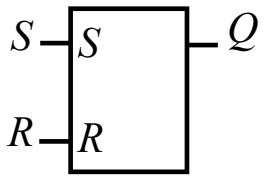
- Definición de su comportamiento mediante una *tabla de transición de estados*

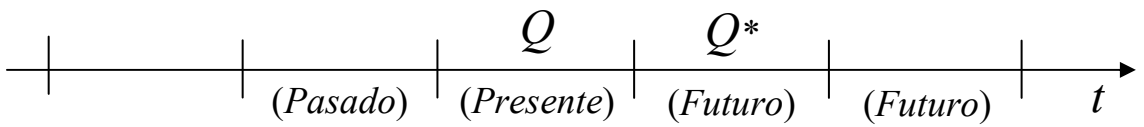
valor de la salida en dos instantes de tiempo distintos

S	R	Q	Q^*	
0	0	0	0	→ *no hagas nada, quédate como estás* $\left(Q^* = Q\right)$.
0	0	1	1	
0	1	0	0	→ *guarda un cero* $(Q^* = 0)$
0	1	1	0	
1	0	0	1	→ *guarda un uno* $(Q^* = 1)$
1	0	1	1	
1	1	0	x	→ *orden prohibida*
1	1	1	x	

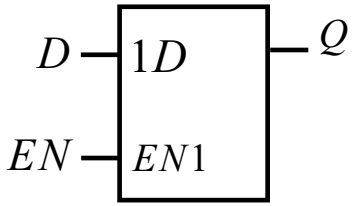
presente (actual)

próximo presente (futuro)



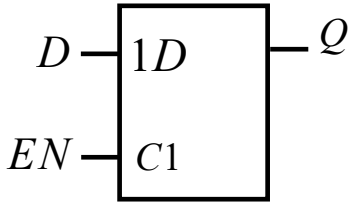


Latch *D* con entrada de habilitación:



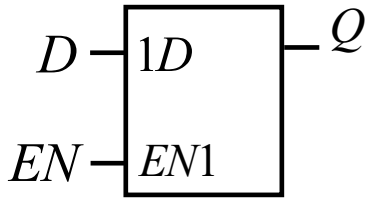
Características:

✓ Tiene una entrada de información (*D*)

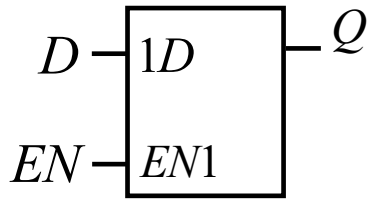


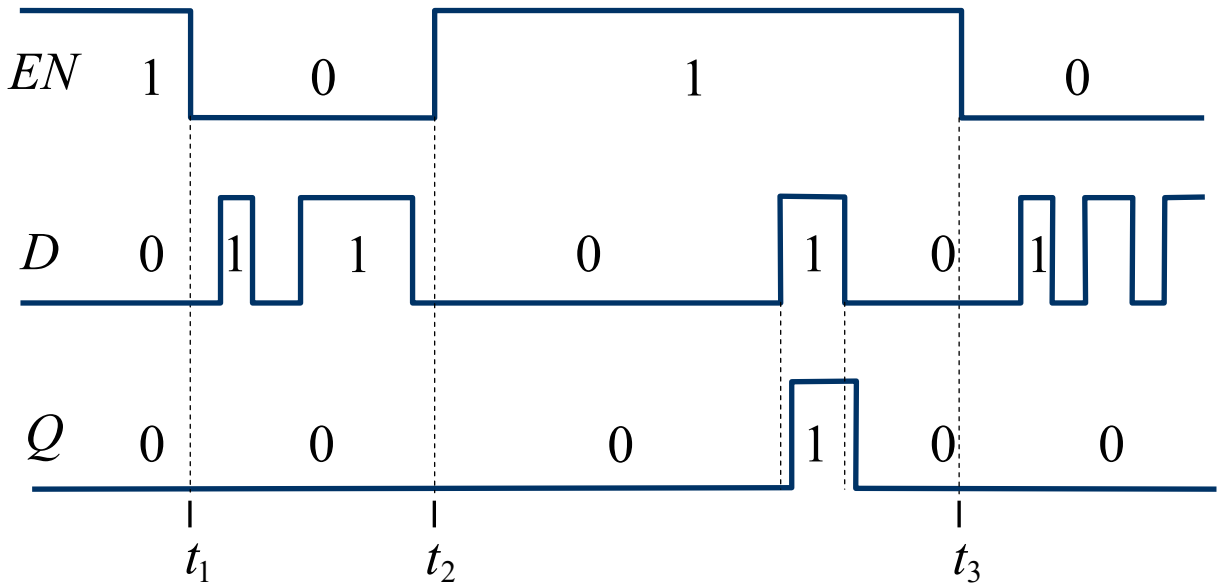
✓ Tiene una entrada auxiliar (*EN*) con la que se establece cuándo el *latch* obedece a la entrada de información y cuándo la ignora

- Definición de su comportamiento mediante una tabla de transición de estados



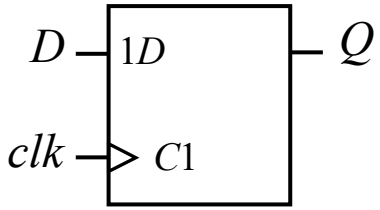
EN	D	Q	Q^*	
0	0	0	0	*no guarda el valor que hay en la entrada de información* ($Q^* = Q$)
0	0	1	1	
0	1	0	0	
0	1	1	1	
1	0	0	0	*guarda el valor que hay en la entrada de información* ($Q^* = D$)
1	0	1	0	
1	1	0	1	
1	1	1	1	





Nota: en la práctica, la salida (Q) actualiza su valor con un pequeño retraso con respecto a los cambios en las entradas.

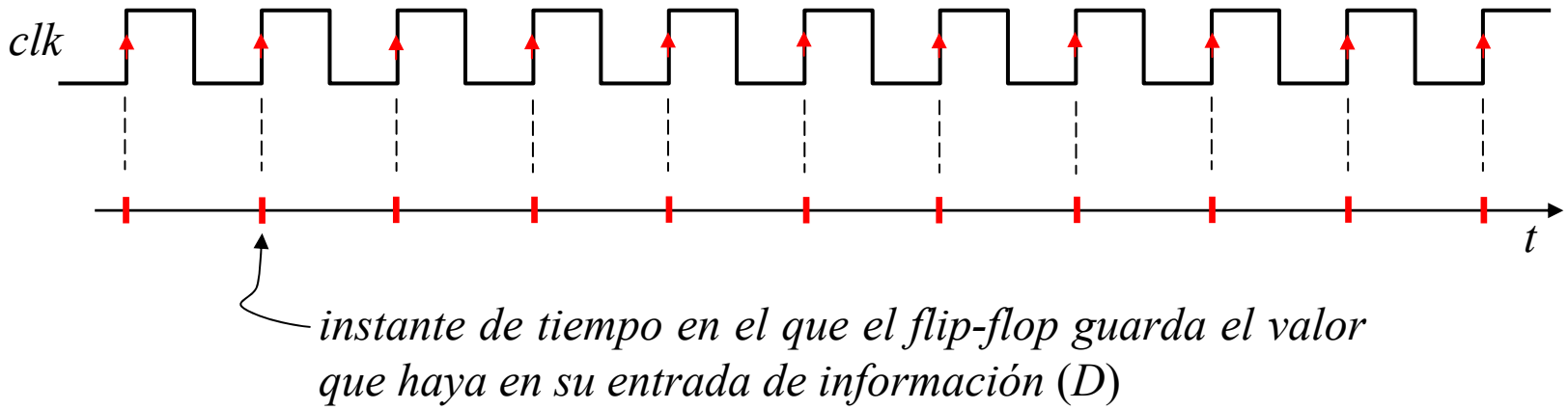
Biestable síncrono: *Flip-Flop D*

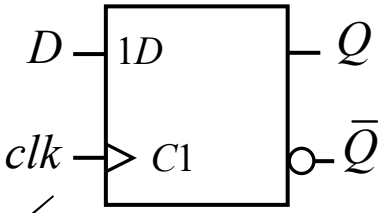


Se caracteriza por lo siguiente:

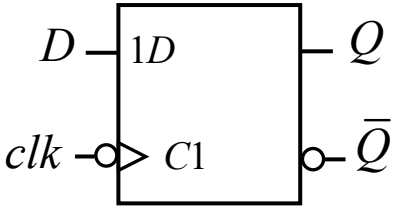
- Tiene 1 entrada de información (*D*).

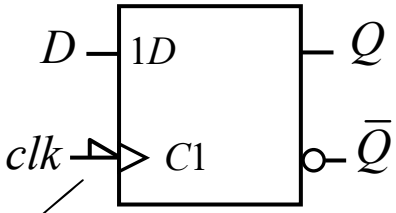
- Tiene 1 entrada de sincronismo (*clk*). La función de dicha entrada es establecer los instantes de tiempo en los que el *flip-flop* guarda el valor existente en su entrada de información (*D*).



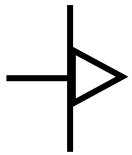


flip-flop D sincronizado con los flancos de subida de la señal de reloj

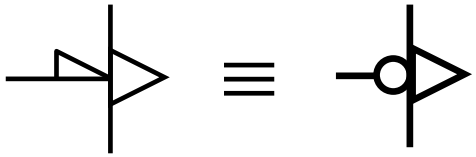




flip-flop D sincronizado con los flancos de bajada de la señal de reloj

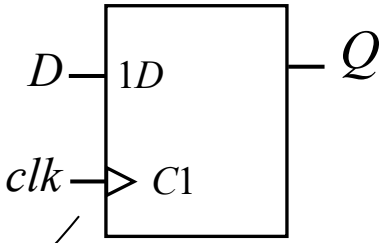


entrada dinámica sensible a los flancos de subida



entrada dinámica sensible a los flancos de bajada

Flip-flop *D* sincronizado con los flancos de subida de *clk*



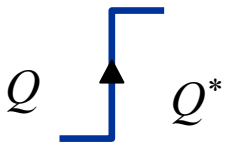
flip-flop D sincronizado con los flancos de subida de clk

valores cuando la señal clk describe un flanco de subida

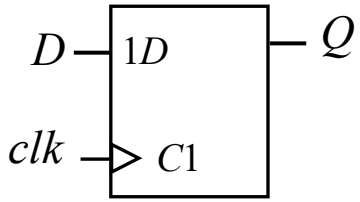
valor guardado por el flip flop después de que la señal clk describa un flanco de subida (es igual al valor que tiene la entrada D durante el flanco de subida de clk)

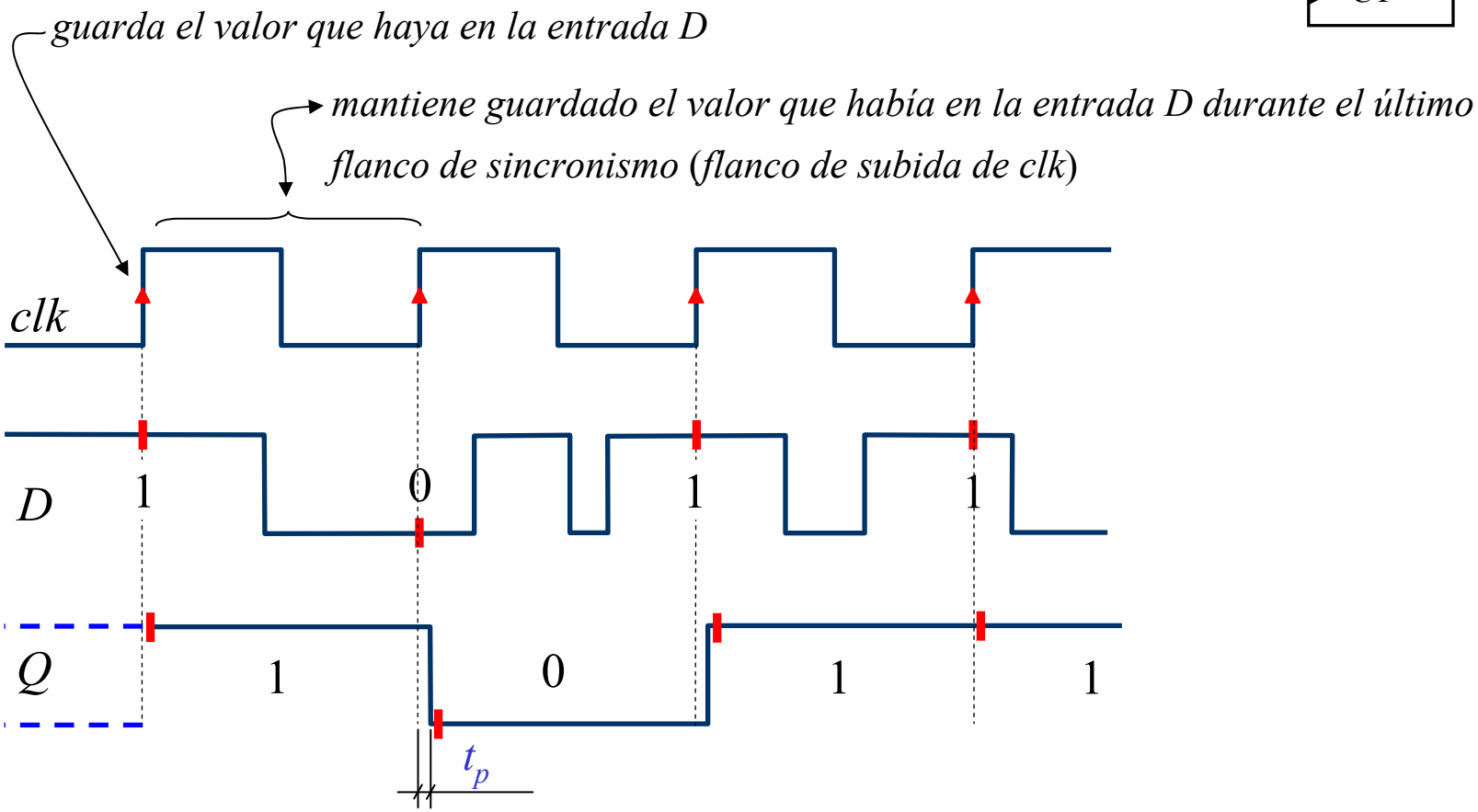
clk	D	Q	Q^*	
↑	0	0	0	*memoriza un cero* $(Q^* = D = 0)$.
↑	0	1	0	
↑	1	0	1	*memoriza un uno* $(Q^* = D = 1)$.
↑	1	1	1	

esta tabla sólo es cierta durante los flancos de subida de la señal de sincronismo (clk)



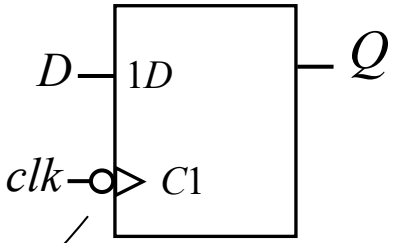
Ejemplo:





Nota: en la práctica, la salida (Q) siempre actualiza su valor con un pequeño retraso t_p con respecto al flanco de sincronismo

Flip-flop *D* sincronizado con los flancos de bajada de *clk*



flip-flop D sincronizado con los flancos de bajada de clk

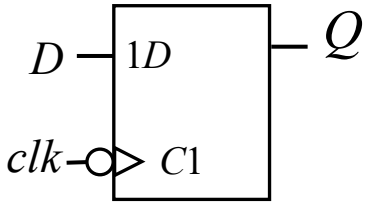
valores cuando la señal clk describe un flanco de bajada

valor guardado por el flip flop después de que la señal clk describa un flanco de bajada (es igual al valor que tiene la entrada D durante el flanco de bajada de clk)

clk	D	Q	Q^*	
↓	0	0	0	*memoriza un cero* $(Q^* = D = 0)$.
↓	0	1	0	
↓	1	0	1	*memoriza un uno* $(Q^* = D = 1)$.
↓	1	1	1	

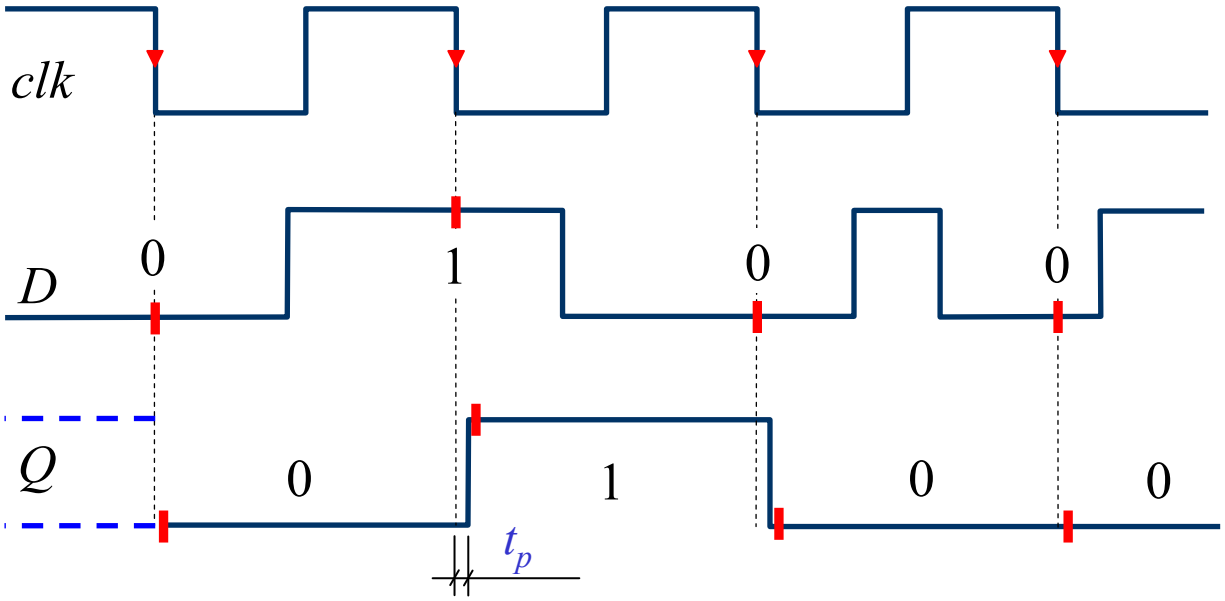
esta tabla sólo es cierta durante los flancos de bajada de la señal de sincronismo (clk)

Ejemplo:



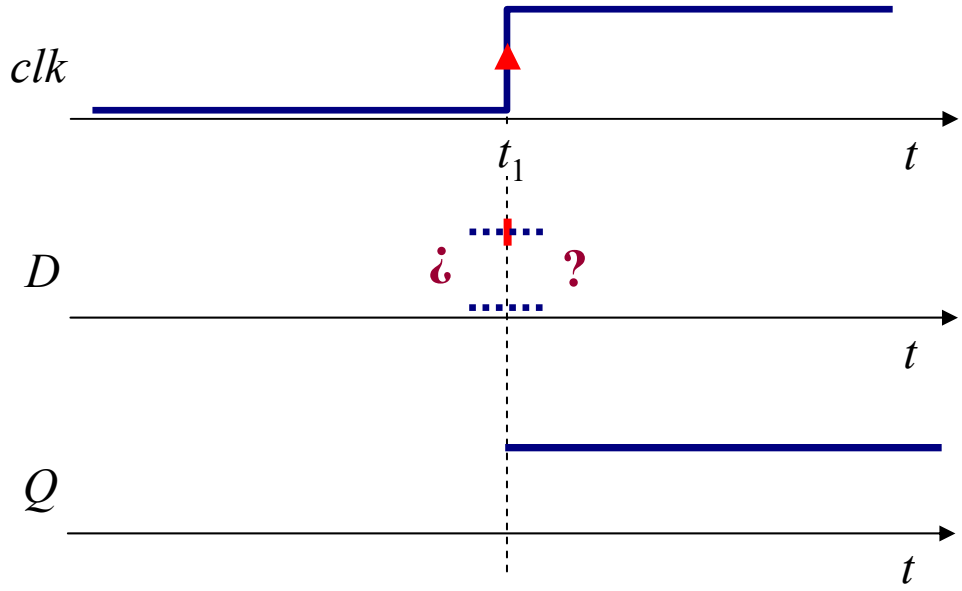
guarda el valor que haya en la entrada D en el instante en el que clk describe el flanco de bajada

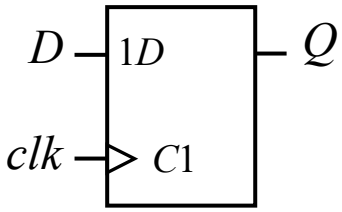
el flip-flop mantiene guardado el valor de la entrada D durante el último flanco de sincronismo (flanco de bajada de clk)



Nota: en la práctica, la salida (Q) siempre actualiza su valor con un pequeño retraso t_p con respecto al flanco de sincronismo

Pregunta: ¿qué valor debe de haber en la entrada de información de un *flip-flop D* durante el instante en el que la señal de reloj (*clk*) describe un flanco de sincronismo, para que en su salida haya un 1 después de que ocurra dicho flanco de sincronismo?





Pregunta: ¿qué valor debe de haber en la entrada de información de un *flip-flop D* durante el instante en el que la señal de reloj (*clk*) describe un flanco de sincronismo, para que en su salida haya un 0 después de que ocurra dicho flanco de sincronismo?

- Procedimiento general de diseño de un sistema secuencial síncrono.

Descripción del funcionamiento del circuito (sist. secuencial) a diseñar

↓

_ Se determina el número de entradas del circuito a diseñar y se le asigna una variable lógica (distinta) a cada una de ellas.

_ Se determina el número de salidas del circuito a diseñar y se le asigna una función lógica (distinta) a cada una de ellas.

(1) *Se dibuja un diagrama de estados que describa el comportamiento del circuito a diseñar.*

camino alternativo

ver ejemplo

Se determina la tabla de transición de estados y la tabla de verdad del circuito combinacional de salida (ccs) a partir del diagrama de estados.

↓

↓

Se codifican los estados del modelo propuesto, asignándole un número binario distinto a cada estado. El número de dígitos de los nombres (≡ números) asignados a los distintos estados del diagrama de estados establece el número de variables de estado que se necesitan para representar el estado (interno) del sistema (n dígitos ⇒ n variables de estado).

Mientras el sistema se encuentre en un determinado estado, las variables de estado (interno) tendrán como valor el número asignado a dicho estado (interno).

↓

Para guardar el valor de cada una de las variables de estado del sistema se utiliza un flip-flop D (n variables de estado ⇒ n flip-flops). El bloque de memoria está formado por todos los flip-flops D utilizados para guardar el valor de las variables de estado ≡ el conjunto de flip-flops que forman el bloque de memoria guardan el valor del estado actual (presente) del sistema.

↓

Se determinan las expresiones más sencillas de las funciones que describen el comportamiento a nivel lógico de los circuitos combinacionales del estado siguiente y de salida.

Dibujar un esquema del circuito.

Simular el funcionamiento del circuito diseñado (*opcional*). *Si el circuito diseñado no funciona de acuerdo con las especificaciones se vuelve al punto* (1)

Construcción de un prototipo. Si su funcionamiento no cumple las especificaciones se vuelve al punto (1)

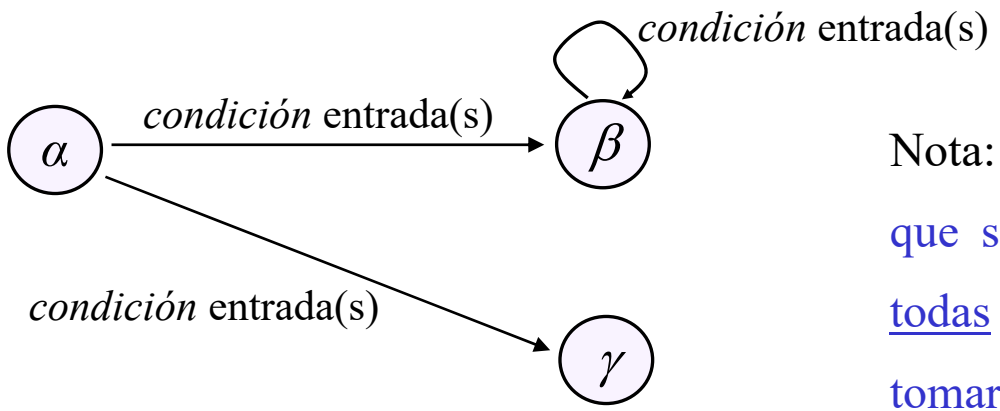
Elementos de un diagrama de estados (*modelo de Moore*)

condición → ≡ *transición* de un estado a otro

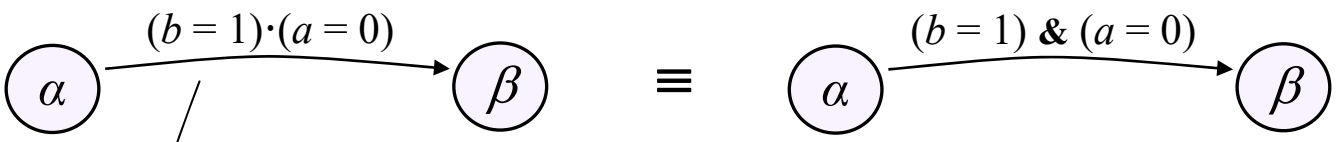
α ≡ *estado* α

valor de las salidas

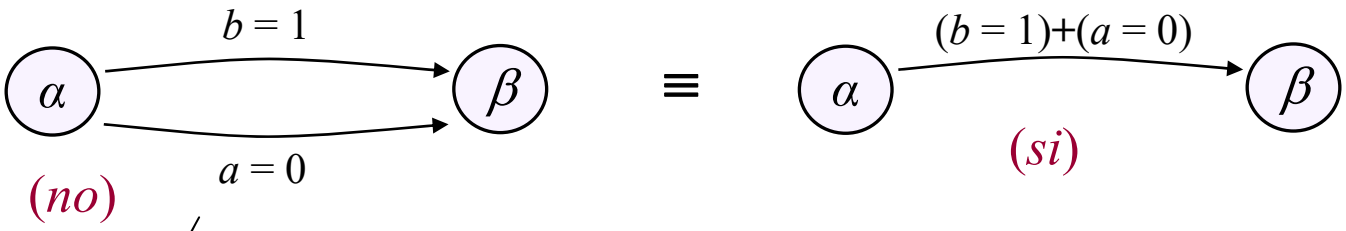
Nota: se puede poner cualquier nombre a un estado durante el dibujo de un diagrama de estados, pero cuando se diseña un circuito que implemente dicho diagrama de estados los nombres de los estados tienen que ser obligatoriamente números binarios.



Nota: las condiciones de las transiciones que salen de un estado deben de abarcar <u>todas</u> los posibles valores que pueden tomar las entradas del sistema.

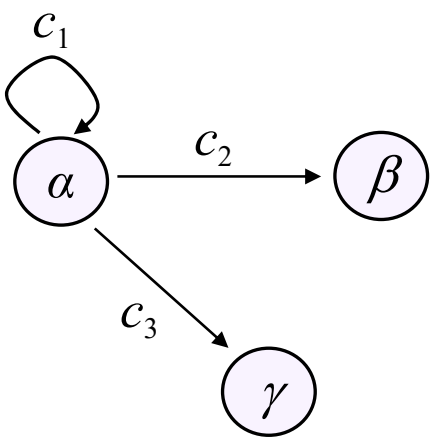


se pasa del estado α al estado β si se cumplen ambas condiciones a la vez



se pasa del estado α al estado β si se cumple al menos una de las condiciones

En un modelo de *Moore*, las salidas están asociadas a los estados. Es decir, en cada estado del diagrama de estados se indica el valor que tiene la salida (o salidas) del sistema secuencial mientras está en dicho estado.



En la figura de la izquierda se indican, a modo de ejemplo, todas las transiciones que salen del estado α.

No se pueden cumplir las condiciones de dos o más transiciones a la vez, pero siempre se tiene que cumplir una de las condiciones (siempre se va a ejecutar una de las transiciones). Es decir, se tiene que cumplir que $c_3 + c_2 + c_1 = 1$.

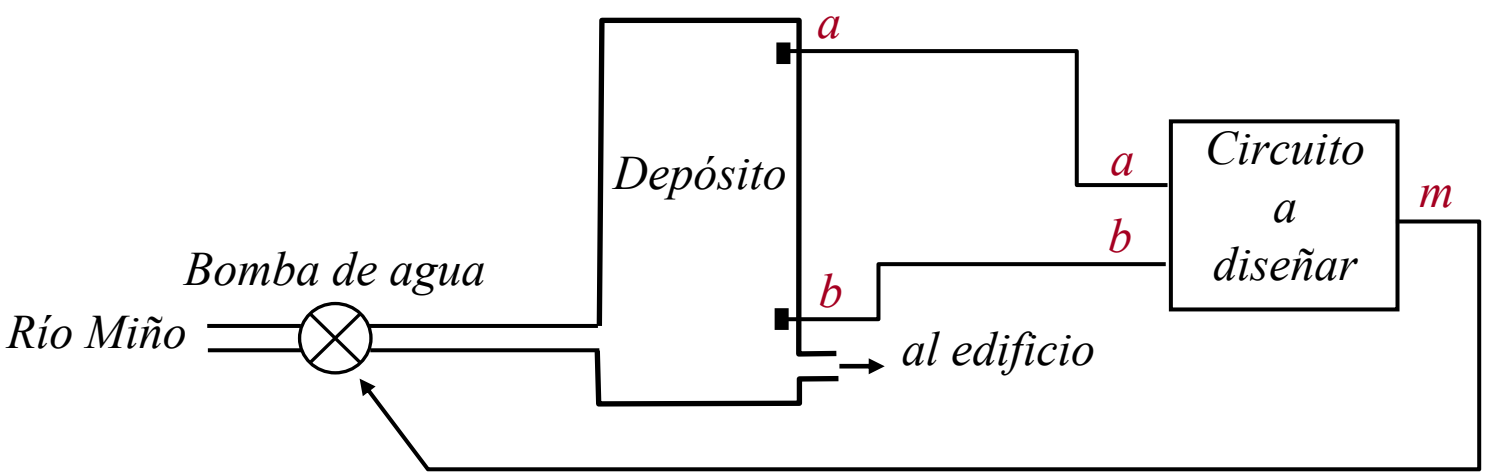
Así, por ejemplo, la condición c_1 se tiene que cumplir cuando no se cumplan las condiciones c_2 y c_3. Es decir, se cumple que,

$$c_1 = \overline{c}_2 \cdot \overline{c}_3 \qquad c_1 = \overline{c_2 + c_3}$$

$$c_2 = \overline{c}_1 \cdot \overline{c}_3$$

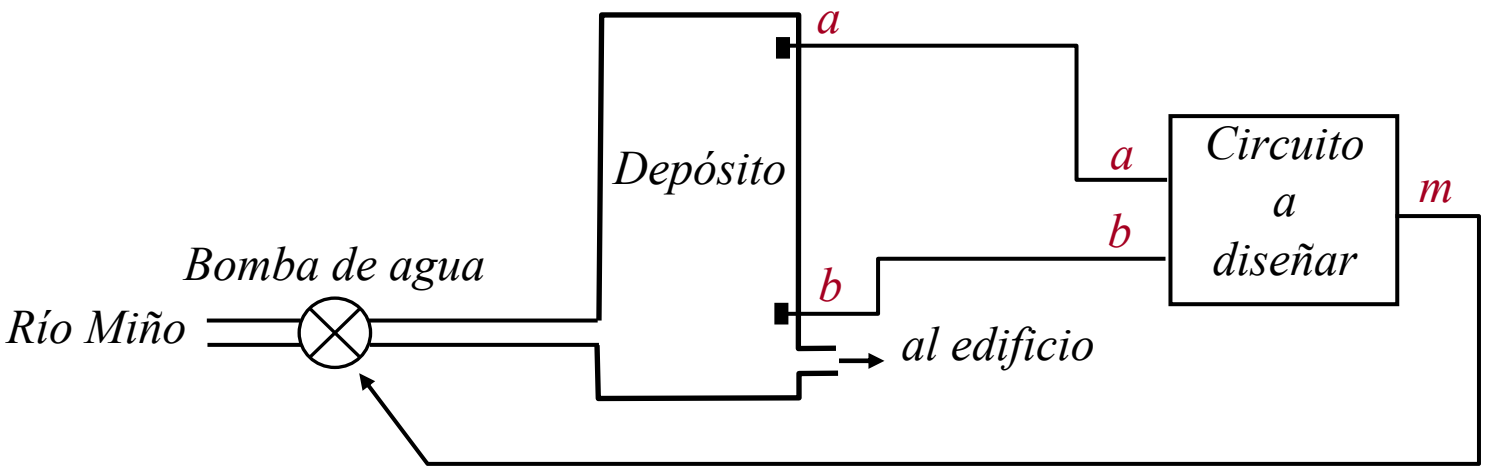
$$c_3 = \overline{c}_1 \cdot \overline{c}_3$$

Ejemplo I *de obtención de un diagrama de estados* (*modelo de Moore*):



Se trata de diseñar un sistema que controle el funcionamiento de la *bomba* (*de agua*), de modo que el nivel del agua en el interior del deposito esté siempre comprendido entre los niveles establecidos por la posición de los sensores *a* y *b*.

Nota: el sistema secuencial que controla la bomba evalúa cada T_{clk} segundos el valor de los sensores *a* y *b* así como el valor del estado interno (valor de la variable o variables de estado). A partir de dicha información actualiza el estado interno (el valor de la variable o variables de estado) y, como consecuencia de ello, el valor de la salida *m*.

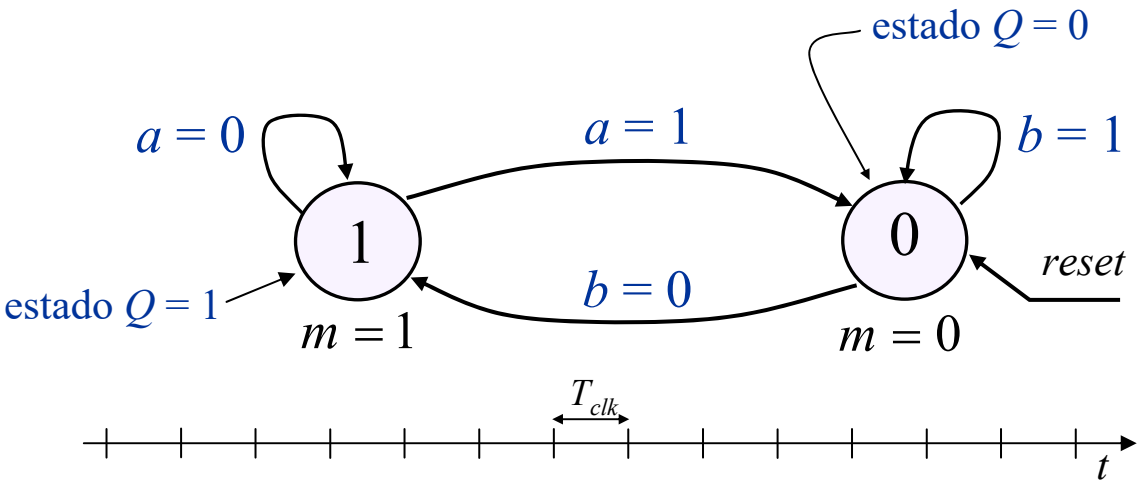


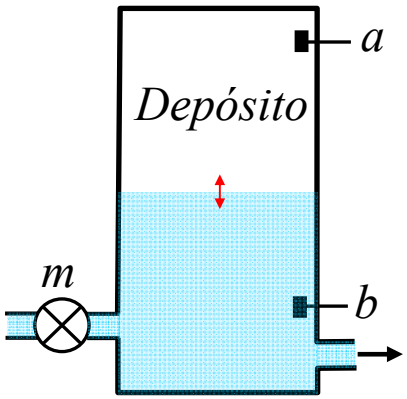
El circuito (sistema secuencial) a diseñar tiene 2 entradas (a y b) y 1 salida (m). Las variables lógicas asignadas a las entradas y la función lógica asignada a la salida se definen de la siguiente manera:

$$a, b = \begin{cases} 1 & \text{si el sensor está en contacto con el agua} \\ 0 & \text{si el sensor no está en contacto con el agua} \end{cases}$$

$$m = \begin{cases} 1 & \text{la bomba bombea agua al interior del depósito} \\ 0 & \text{la bomba está parada} \end{cases}$$

Diagrama de estados o de flujo (*modelo de Moore*):

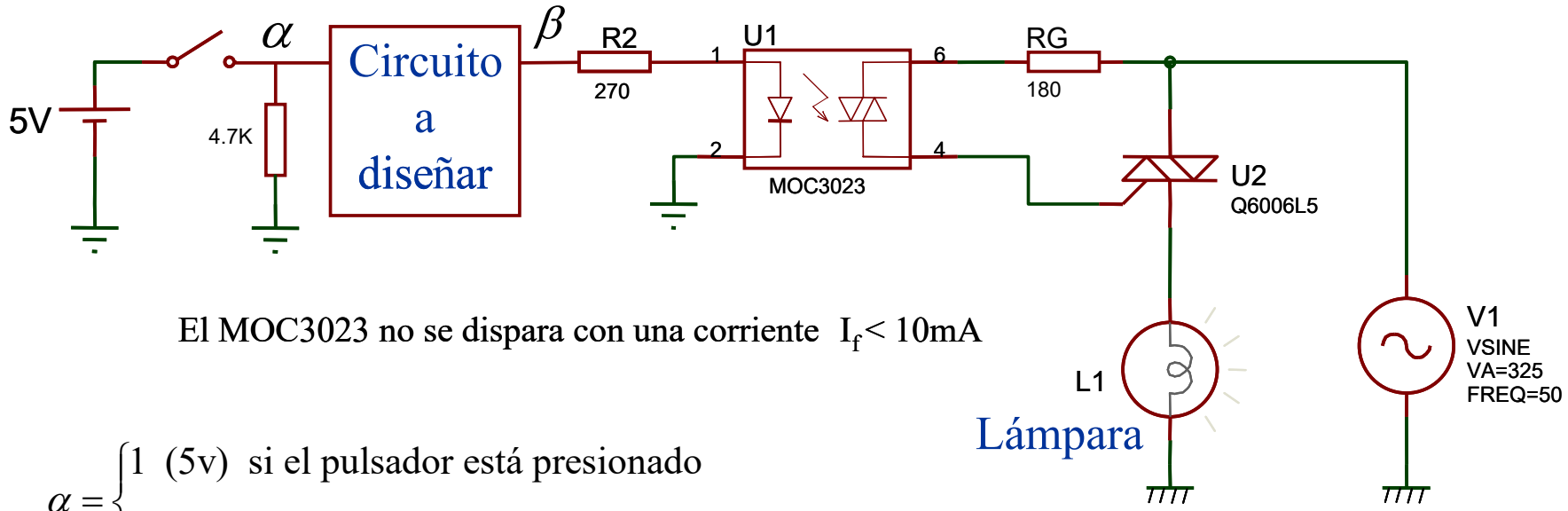




En el estado $Q = 0$, la bomba de agua está parada ($m = 0$) y el depósito se está vaciando. El sistema se mantiene en el estado $Q = 0$ mientras el nivel del agua sea superior al nivel en el que se encuentra el sensor b. Cuando el nivel del agua desciende por debajo de la posición del sensor b ($b = 0$), el sistema pasa al estado $Q = 1$ en el que se bombea agua al interior del deposito ($m = 1$). El sistema se mantiene en el estado $Q = 1$ hasta que el nivel del agua en el interior del depósito alcanza el nivel en el que se encuentra el sensor a ($a = 1$), en cuyo momento el sistema pasa al estado $Q = 0$, en el que la bomba de agua está parada ($m = 0$).

Ejemplo II *de obtención de un diagrama de estados* (*modelo de Moore*):

En este caso se trata de diseñar un sistema de control que haga que cada vez que se presione un pulsador α se encienda una lámpara, en el caso de que esté apagada, o se apague en el caso de que esté encendida. "*La lámpara cambia de estado* (*encendida/apagada*) *cada vez que se deja de presionar el pulsador* (*botón*) α".



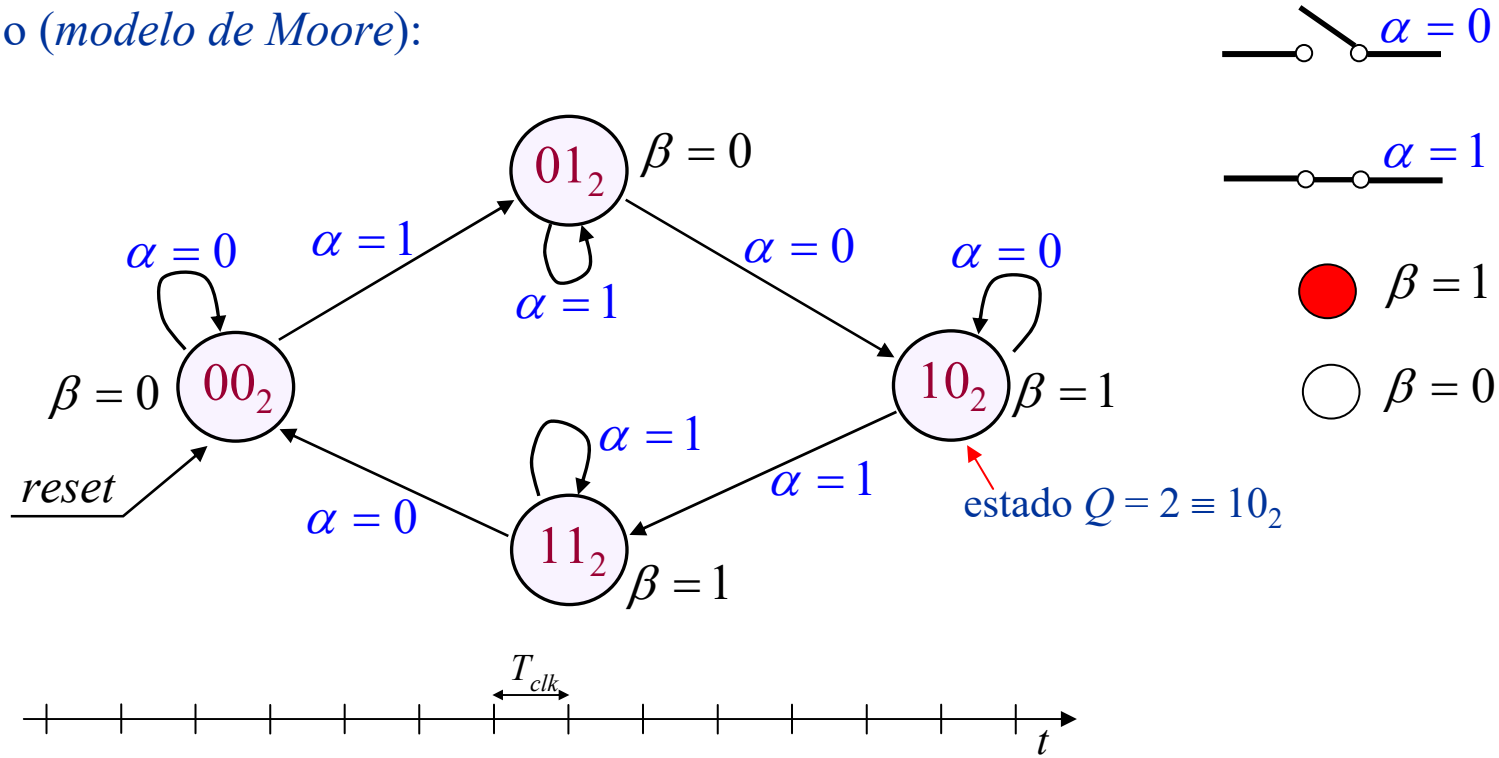
El MOC3023 no se dispara con una corriente $I_f < 10mA$

$$\alpha = \begin{cases} 1 \ (5v) \ \text{si el pulsador está presionado} \\ 0 \ (0v) \ \text{si el pulsador no está presionado} \end{cases}$$

Nota: pulsador ≠ interruptor

$$\beta = \begin{cases} 1 \ \text{la bombilla está encendida} \\ 0 \ \text{la bombilla está apagada} \end{cases}$$

Diagrama de flujo (*modelo de Moore*):



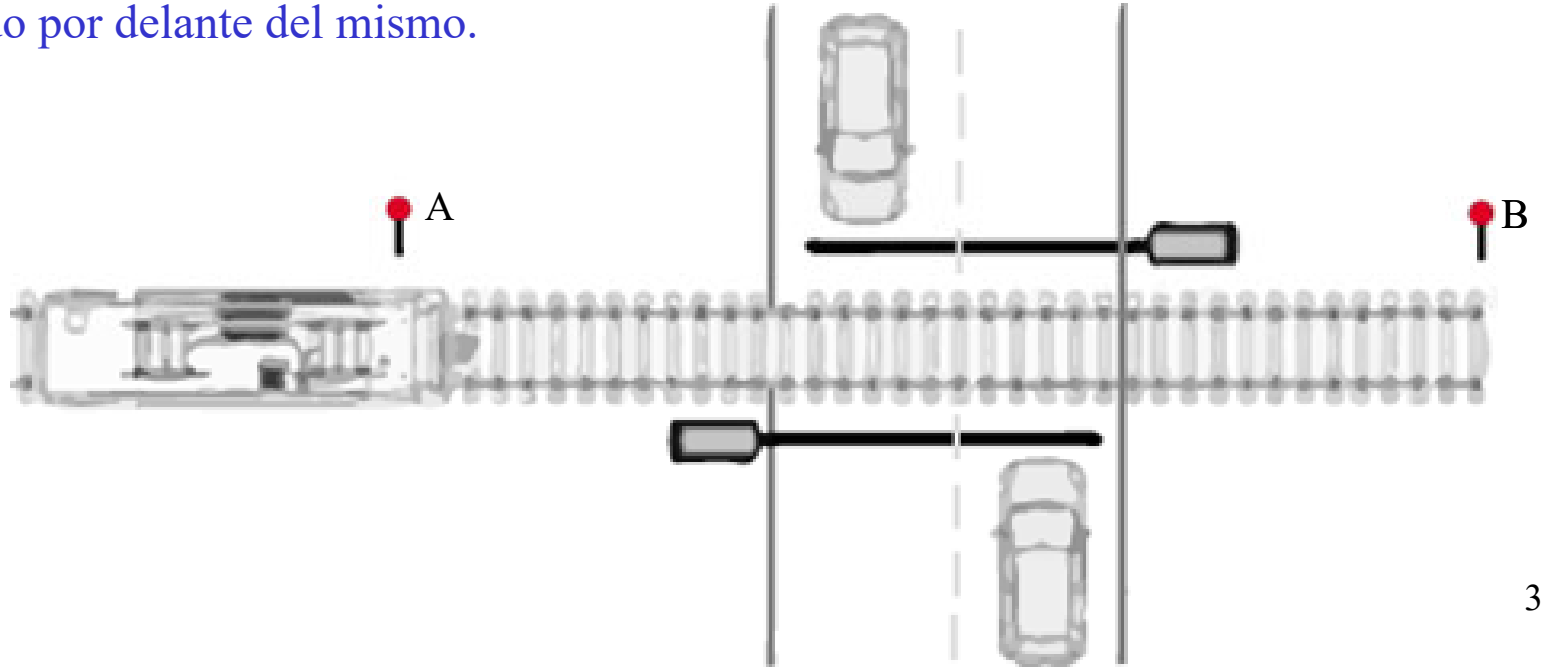
En el *estado* 0, la bombilla está apagada ($\beta = 0$) y el pulsador no está presionado ($\alpha = 0$). Estando en este estado, si se presiona el pulsador ($\alpha = 1$), el sistema pasa al *estado* 1, en el que permanece hasta que se deja de presionar el pulsador ($\alpha = 0$), en cuyo momento el sistema pasa al *estado* 2. En dicho estado la bombilla está encendida ($\beta = 1$) y el pulsador no está presionado ($\alpha = 0$). Estando en el estado 2, si se presiona el pulsador el sistema pasa al estado 3, en el que permanece hasta que se deja de presionar el pulsador. En cuyo momento el sistema pasa al estado 0, en el que la bombilla permanece apagada.

Ejemplo III *de obtención de un diagrama de estados* (*modelo de Moore*):

En este caso se trata de determinar un diagrama de estados que modele el sistema de control de las barreras de un paso a nivel. A la hora de determinar el diagrama de estados hay que tener en cuenta lo siguiente:

▪ Hay dos sensores (A y B) sobre la vía del tren, situados cada uno a un lado de la carretera que atraviesa la vía.

▪ Cada sensor (A, B) tiene una salida binaria en la que hay un 1 lógico siempre que un tren esté pasando por delante de dicho sensor y hay un 0 lógico siempre que no hay un tren pasando por delante del mismo.

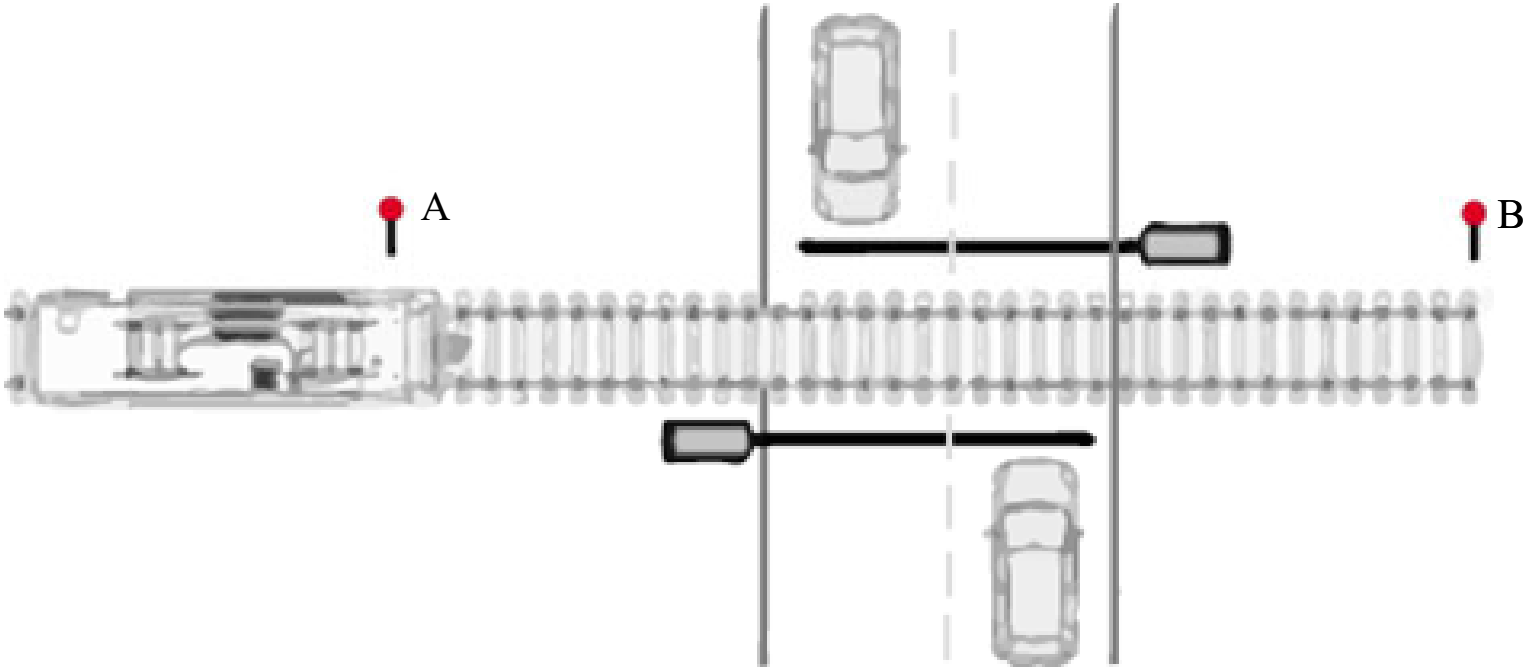


▪ La posición de las barreras se controla mediante una señal binaria u que cumple lo siguiente:

$u = 1 \rightarrow$ las barreras están subidas

$u = 0 \rightarrow$ las barreras están bajadas

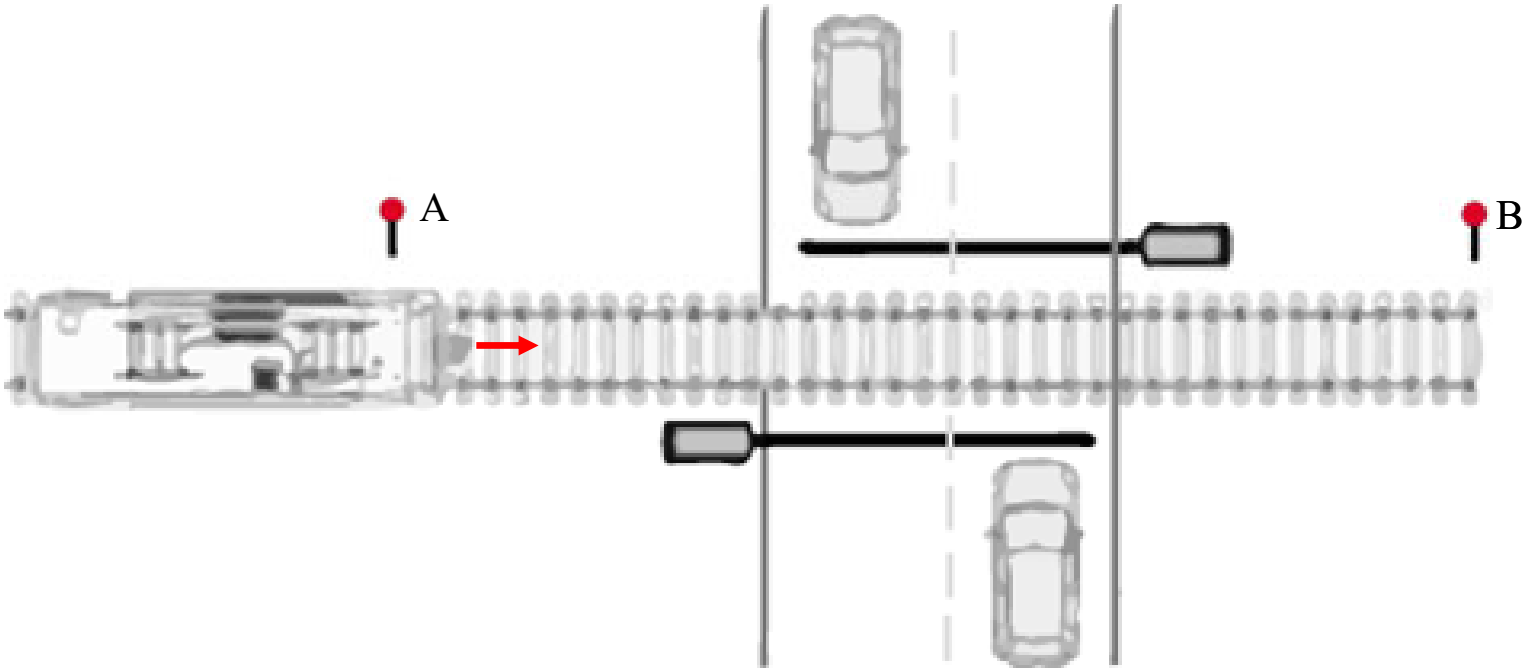
▪ Las barreras deben estar bajadas ($u = 0$) siempre que haya al menos una parte de un tren en el trayecto de la vía férrea delimitado por los sensores A y B. Las barreras no se pueden subir (permitiendo la circulación de los coches) hasta que todo el tren se encuentre fuera del tramo comprendido entre las posiciones de los sensores A y B.



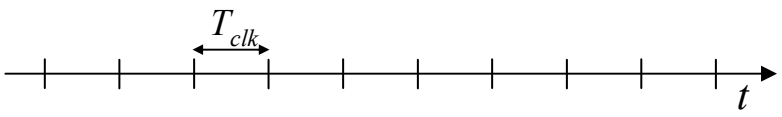
▪ La velocidad del tren está limitada en el tramo comprendido entre las posiciones en las que se encuentran los sensores (A y B) de modo que, cuando el tren se aproxima al paso a nivel, el tiempo que este tarda en recorrer la distancia comprendida entre el sensor A y el paso a nivel es mucho mayor que el tiempo que tardan en bajarse las barreras.

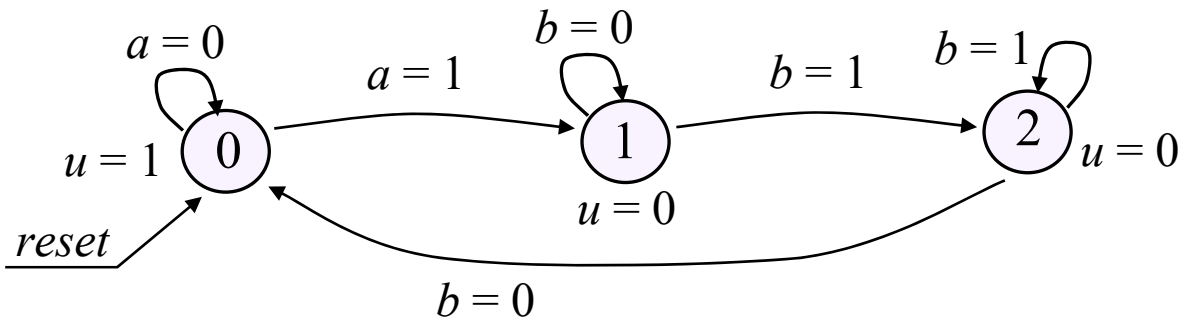
▪ La longitud de un tren (locomotora + vagones) puede ser mayor o menor que la distancia que separa a los sensores A y B.

▪ Los trenes sólo pueden circular en un sentido por la vía férrea considerada (de izquierda a derecha en el siguiente esquema).



Un diagrama de estados (modelo de *Moore*)

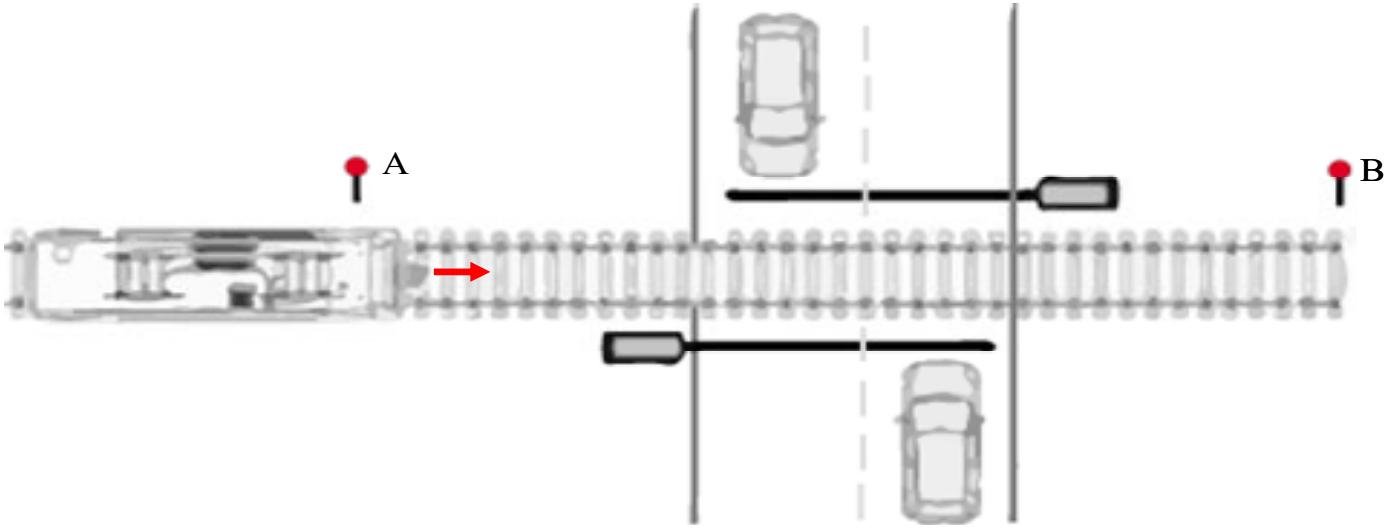


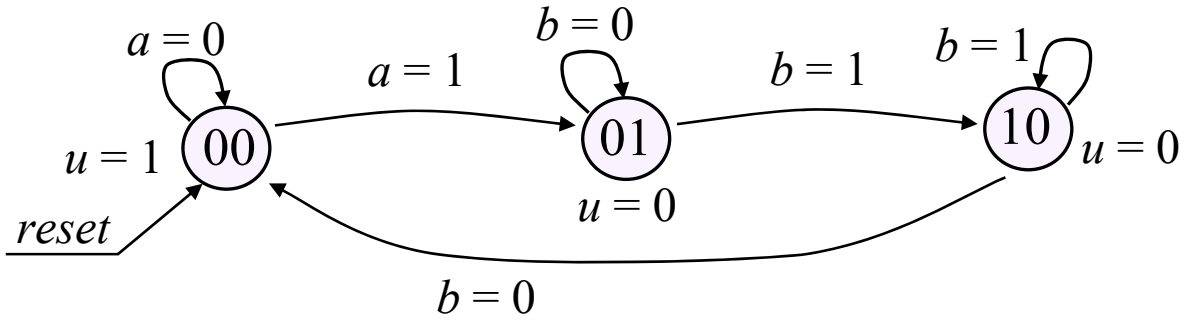


estado 0: las barreras están subidas (no hay un tren circulando por las vías en el tramo comprendido entre las posiciones de los sensores A y B)

estado 1: un tren está pasando por delante del sensor A (barreras bajadas)

estado 2: un tren está pasando por delante del sensor B (barreras bajadas)

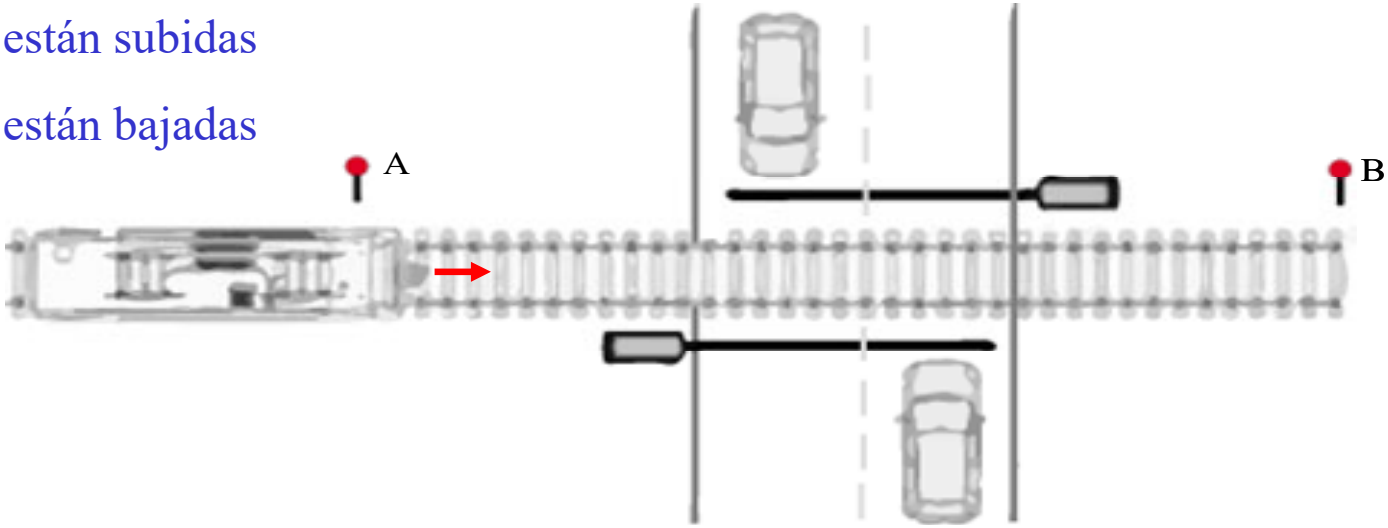




$$a = \begin{cases} 1 & \text{si el tren está pasando por delante del sensor A} \\ 0 & \text{si no es así} \end{cases}$$

$$b = \begin{cases} 1 & \text{si el tren está pasando por delante del sensor B} \\ 0 & \text{si no es así} \end{cases}$$

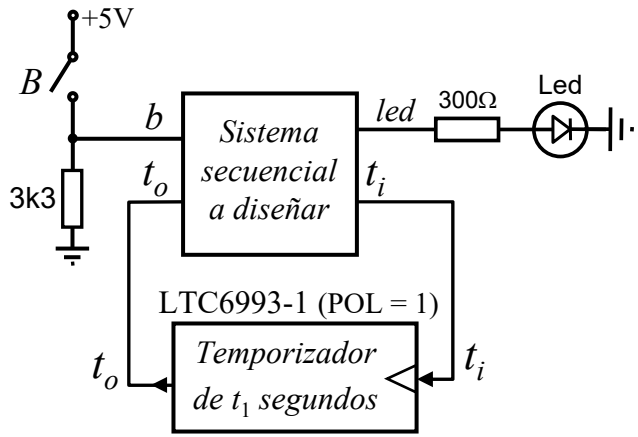
$$u = \begin{cases} 1 & \text{las barreras están subidas} \\ 0 & \text{las barreras están bajadas} \end{cases}$$



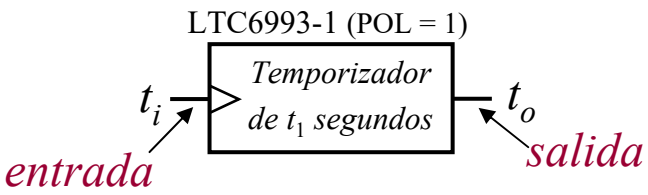
Ejemplo IV *de obtención de un diagrama de estados* (*modelo de Moore*):

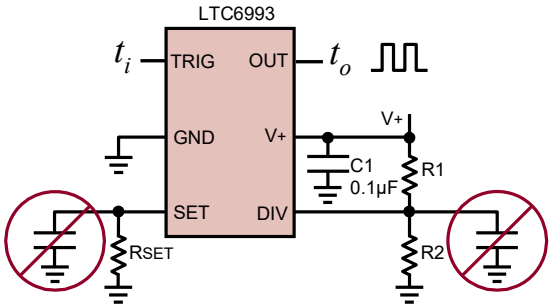
En este caso se trata de determinar un diagrama de estados que modele un sistema que controla el encendido/apagado de un *led* (≡ diodo emisor de luz). En concreto, el funcionamiento del sistema a modelar se caracteriza por lo siguiente: "*estando el led apagado, cada vez que se pulsa el botón B* (b = 1), *se debe de encender el led* (*led* = 1), *debiendo de permanecer encendido durante* t_1 *segundos. Mientras el led está encendido, si se pulsa el botón B no debe tener ningún efecto sobre el estado* (*encendido/apagado*) *del led*"

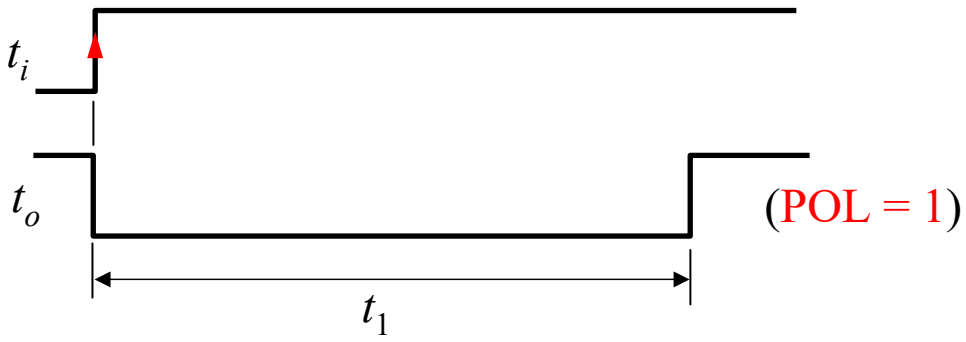
Nota: a la hora de plantear el diagrama de estados se debe considerar que se dispone de un temporizador (como el LTC6993-1) ajustado para realizar temporizaciones de t_1 seg. (ver pág. siguiente) y que $T_{clk} <<< t_1$



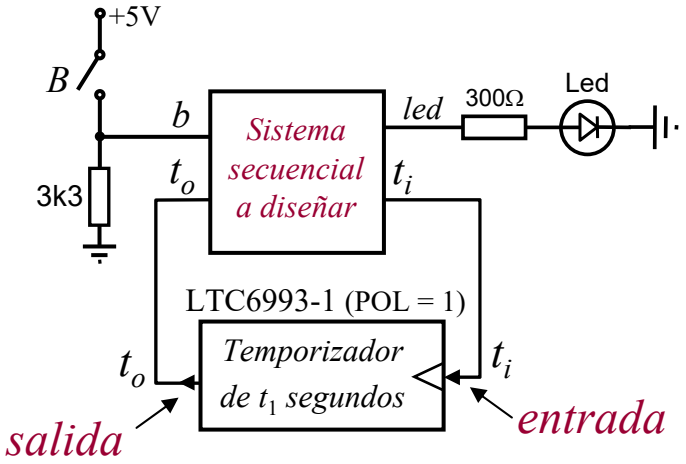
Comportamiento de un monoestable LTC6993-1





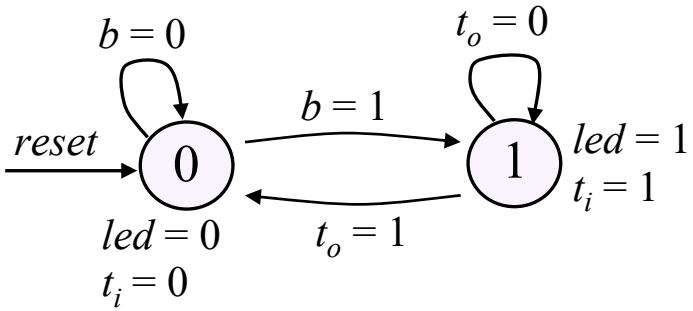


Nota: el temporizador LTC6993-1 se puede configurar para realizar temporizaciones entre 1µseg y 33,6seg. No es redisparable



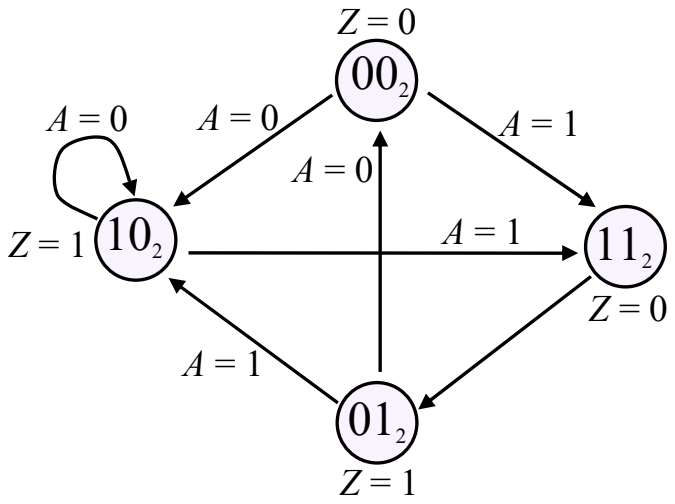
$$b = \begin{cases} 1 & \text{el botón } B \text{ está pulsado} \\ 0 & \text{si no es así} \end{cases}$$

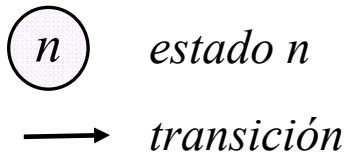
$$led = \begin{cases} 1 & \text{el diodo } led \text{ está encendido} \\ 0 & \text{si no es así} \end{cases}$$



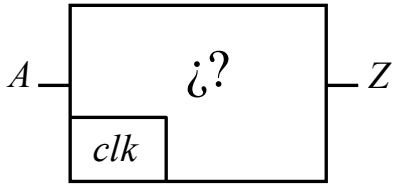
Nota: el sistema secuencial, además de controlar el estado del *led*, también controla el funcionamiento del temporizador LTC6993-1.

Ejemplo de implementación de un diagrama de estados (modelo de Moore)



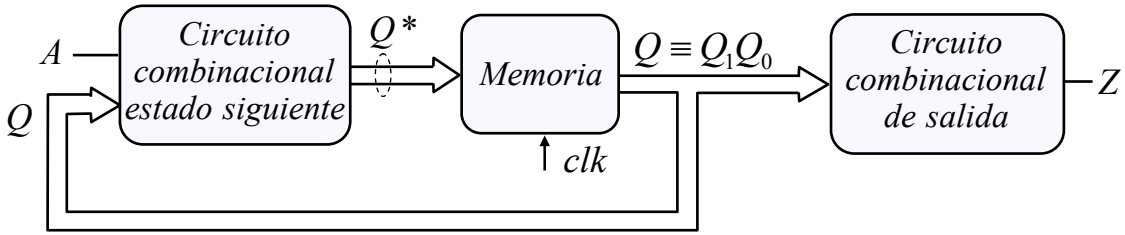


Del diagrama de flujo se deduce lo siguiente:

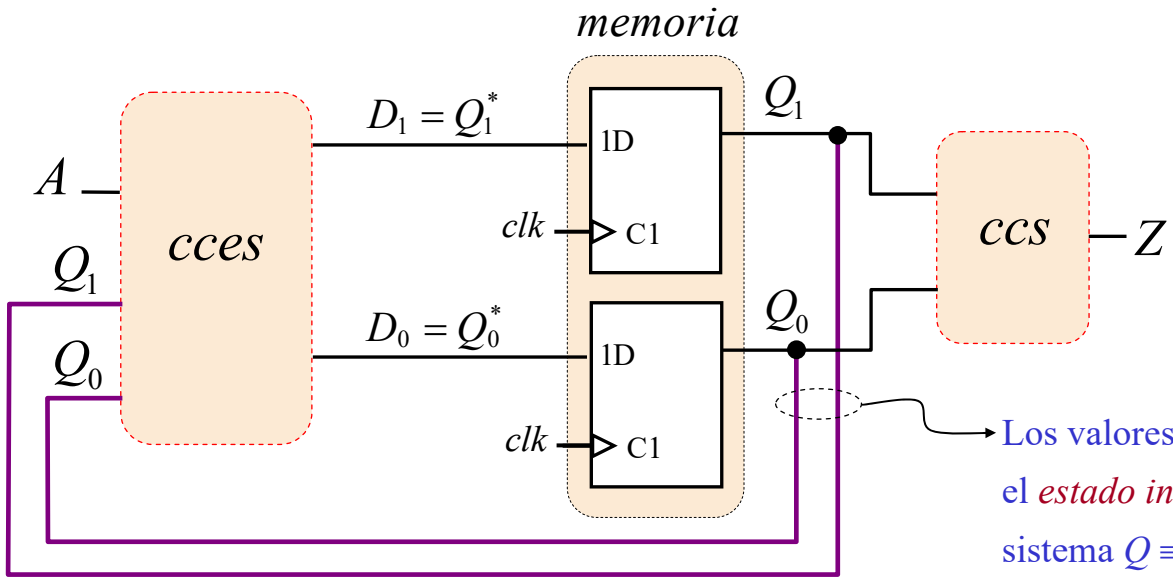


- *tiene una única entrada*: A
- *tiene una única salida*: Z
- *tiene* 4 *estados, los cuales se han codificado asignándole a cada uno un número binario de* 2 *dígitos. De acuerdo con esto, el sistema tendrá* 2 *variables de estado* (Q_1, Q_0) *y, por lo tanto, el bloque de memoria estará formado por* 2 *flip-flops D para guardar los valores de las variables de estado.*

De acuerdo con el diagrama de estados de la página anterior, el diagrama de bloques del circuito (sistema secuencial) a diseñar es el siguiente:



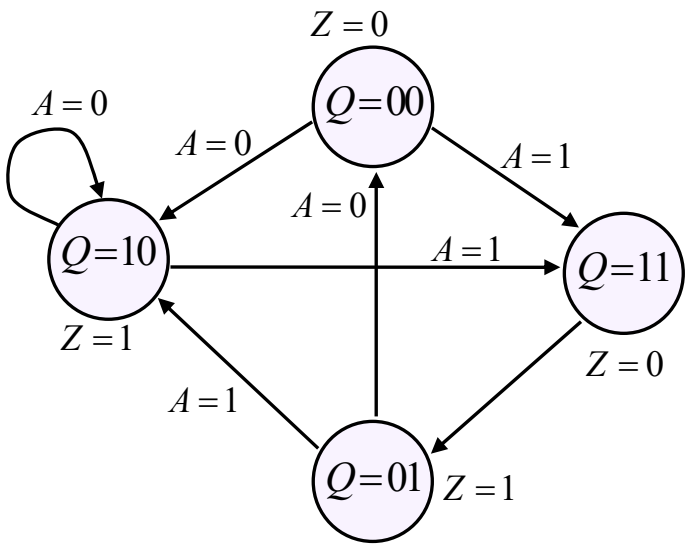
≡



Los valores de Q_1 y de Q_0 constituyen el *estado interno* actual o presente del sistema $Q \equiv (Q_1, Q_0)$.

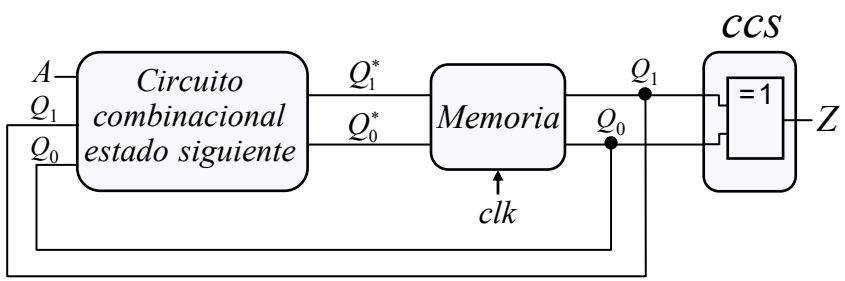
Pregunta: ¿Por qué el sistema sólo puede cambiar de estado durante los flancos de subida de la señal *clk*?

Diseño del circuito que genera la salida del sistema (*ccs*): a partir del diagrama de estados se determina la *tabla de verdad* de la función $Z(Q_1,Q_0)$ implementada por el *ccs*, cumpliéndose que:

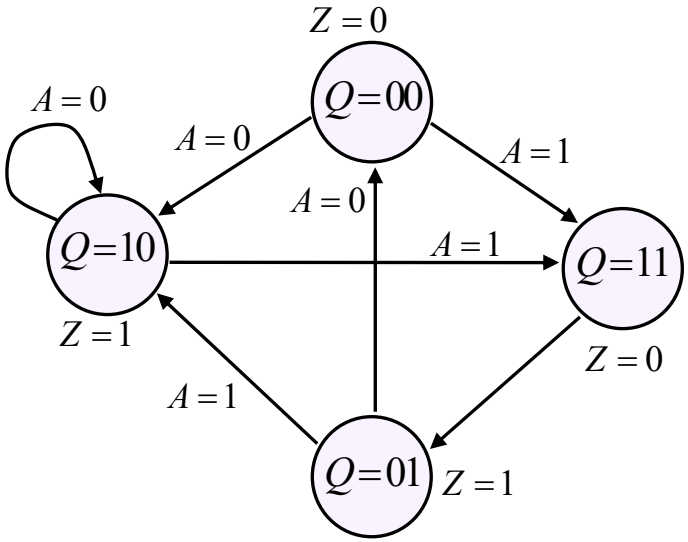


Q_1	Q_0	Z
0	0	0
0	1	1
1	0	1
1	1	0

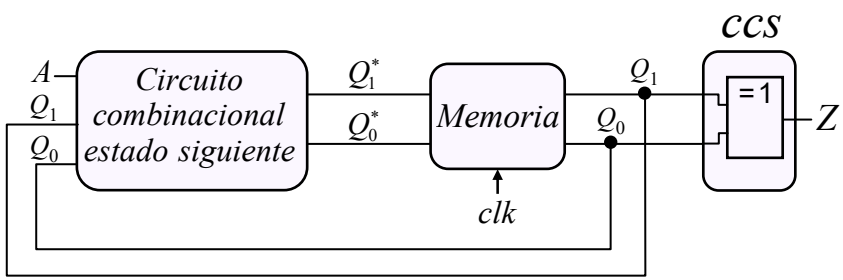
$$z(Q_1,Q_0) = \sum_2(1,2) = \overline{Q}_1 Q_0 + Q_1\overline{Q}_0 = Q_1 \oplus Q_0$$



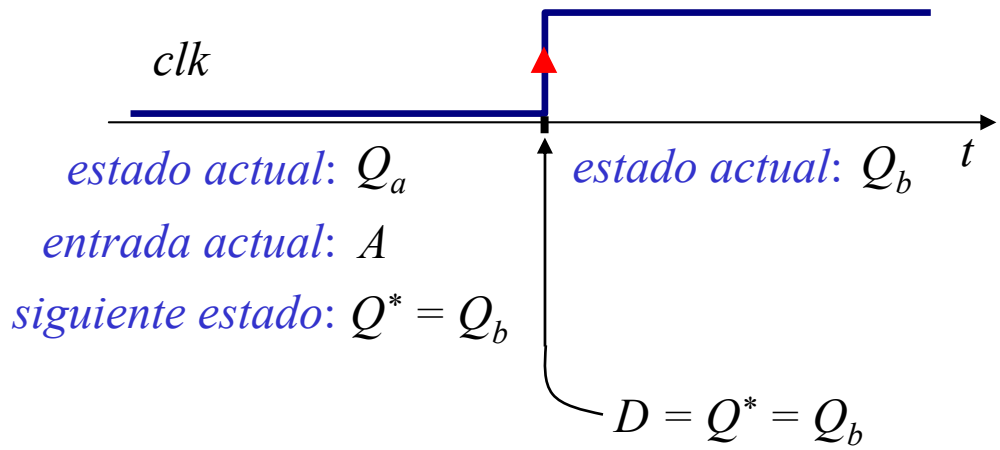
Diseño del circuito que determina el siguiente estado interno (*cces*): a partir del diagrama de estados se determina la siguiente *tabla de transición de estados*

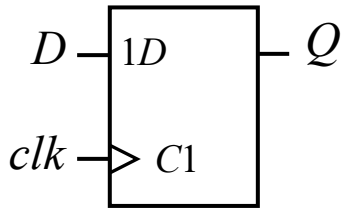


Q			Q^*	
Q_1	Q_0	A	Q_1^*	Q_0^*
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1
presente			*futuro*	



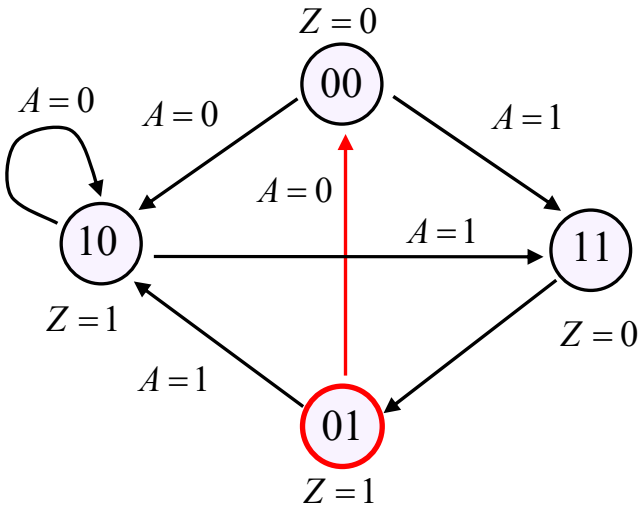
La tabla de *transición de estados* indica para cualquier estado en el que se encuentre el sistema, cual será el estado del sistema a partir del próximo flanco de sincronismo de la señal de reloj (*clk*). Dicho siguiente estado será el valor que guardarán los *flip-flops D* que forman el bloque de memoria a partir del próximo flanco de sincronismo de la señal de reloj.



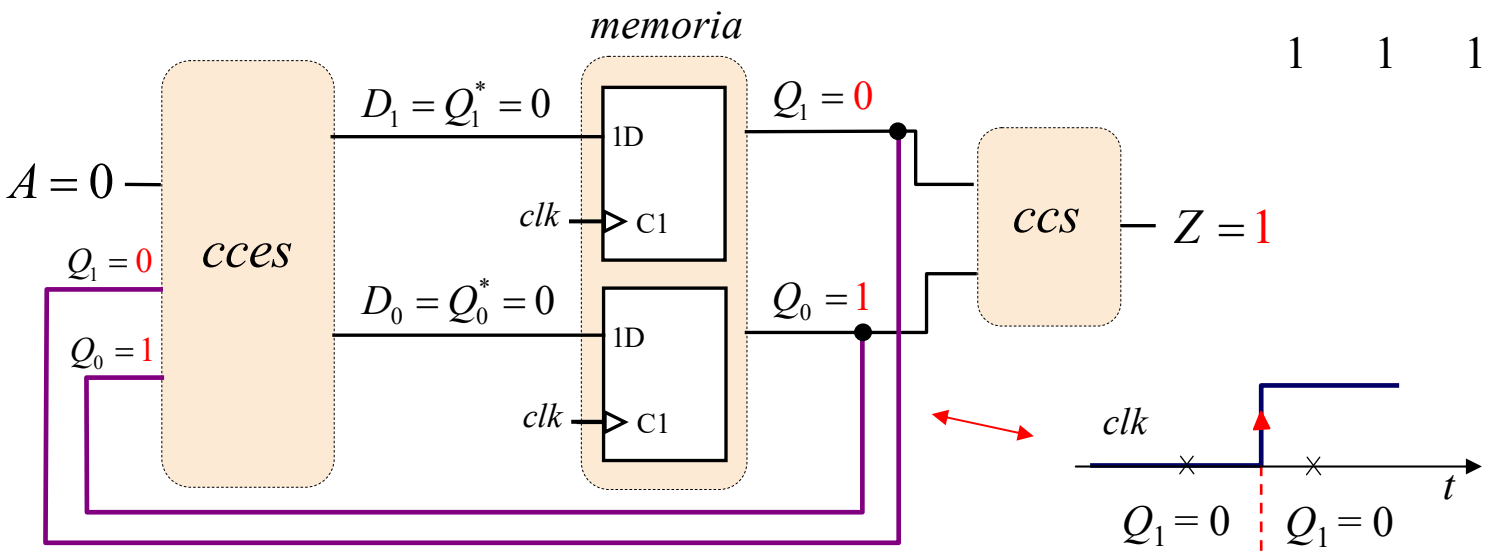


Q			Q^*	
Q_1	Q_0	A	Q_1^*	Q_0^*
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1
presente			*futuro*	

Ejemplo:



Q_1	Q_0	A	Q_1^*	Q_0^*
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1

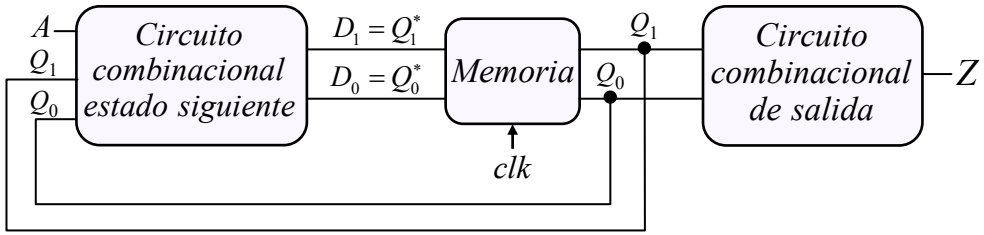


Esto hace que los valores Q_1^* y Q_0^* que proporciona la tabla de transición de estados sean los valores (D_1 y D_0) que debe de haber en las salidas del circuito combinacional que genera el siguiente estado (*cces*) antes y durante el próximo flanco de sincronismo de la señal de reloj (*clk*). Es decir, se cumple que:

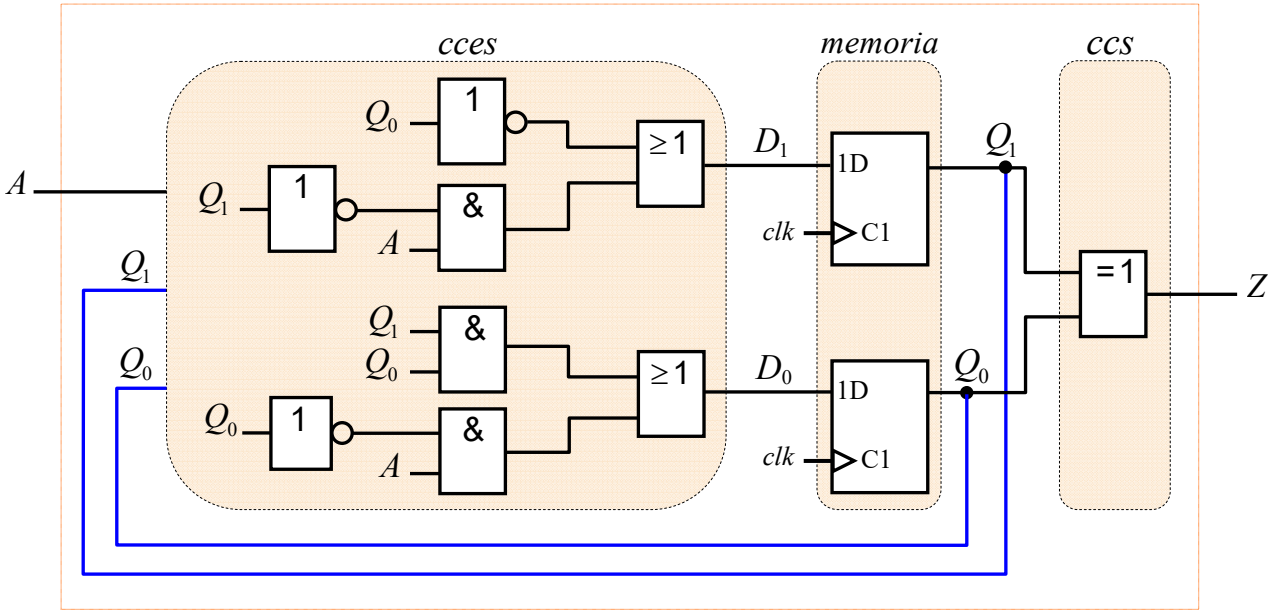
$$D_1(Q_1,Q_0,A) = Q_1^*(Q_1,Q_0,A) = \sum\nolimits_3(0,1,3,4,5) = \overline{Q}_1 A + \overline{Q}_0$$

$$D_0(Q_1,Q_0,A) = Q_0^*(Q_1,Q_0,A) = \sum\nolimits_3(1,5,6,7) = \overline{Q}_0 A + Q_1 Q_0$$

Q			Q^*	
Q_1	Q_0	A	Q_1^*	Q_0^*
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1



De acuerdo con los resultados anteriores, el circuito que implementa el diagrama de estados es el siguiente



$$D_1(Q_1, Q_0, A) = \bar{Q}_1 A + \bar{Q}_0$$

$$D_0(Q_1, Q_0, A) = \bar{Q}_0 A + Q_1 Q_0$$

$$Z(Q_1, Q_0) = Q_1 \oplus Q_0$$

* *clk* es una señal auxiliar que sincroniza el funcionamiento de los *flip - flops*

Parte II del Tema 5